

Partiel de Programmation 4

Mardi 19 mai 2026 de 9 heures à 11 heures

AVERTISSEMENT

Remplissez l'encadré ci-dessous avec vos nom et prénom (en lettres MAJUSCULES) et votre numéro d'étudiant(e). Dès que vous serez autorisé(e) à ouvrir le livret, commencez IMPÉRATIVEMENT par lire l'exercice 0 à la page 2 du livret !

NOM :

PRÉNOM :

N° d'étudiant(e) :

Barème provisoire sur 22 points :

Ex. 1 :	1.A. / 2 pt	1.B. / 1 pt				/ 3 pt
Ex. 2 :	2.A. / 1 pt	2.B. / 2 pt	2.C. / 1 pt			/ 4 pt
Ex. 3 :	3.A. / 1 pt	3.B. / 3 pt	3.C. / 3 pt	3.D. / 3 pt	3.E. / 1 pt	/ 11 pt
Ex. 4 :	4.A. / 1 pt	4.B. / 2 pt	4.C. / 1 pt			/ 4 pt
Total :						/ 22 pt

Exercice 0. Consignes et notations

Vérifiez que vos nom et prénom ont bien été écrits en **LETTRES MAJUSCULES** dans l'encadré de la première page.

Ce livret de 15 pages est la copie d'examen : traitez chaque question dans le cadre réservé à la réponse, sans déborder. Ne dégrafez pas le livret !

N.B. Le nombre de lignes dédiées aux réponses est plus que suffisant !

Lisez tout l'énoncé avant de commencer à répondre aux questions et gardez quelques minutes pour vous relire avant la fin de l'épreuve.

Rédigez vos réponses au brouillon avant de les reporter à l'encre sur ce livret. Soyez rigoureux : soignez la présentation, écrivez lisiblement et codez de façon précise. Les réponses ambiguës ou rendues difficilement lisibles à cause d'une écriture trop peu soignée ou d'une surcharge de ratures seront considérées comme fausses.

Respectez SCRUPULEUSEMENT l'énoncé : ne modifiez pas les noms des fonctions ni ceux des paramètres et types structurés quand ils sont précisés.

Les seuls types structurés que vous êtes autorisé(e) à utiliser sont les suivants

```
1 struct node {  
2     int label; /* étiquette du noeud */  
3     struct node * left; /* lien vers le sous-arbre gauche */  
4     struct node * right; /* lien vers le sous-arbre droit */  
5 };  
6 typedef struct node * link; /* lien vers (la racine d')un arbre binaire */
```

Les définitions des fonctions qu'on vous demande d'écrire **ne peuvent faire appel qu'à des fonctions dont la définition a fait l'objet d'une question antérieure**.

Pour répondre à une question, **vous pouvez toujours considérer comme acquises les réponses aux questions antérieures**.

Bon travail !

Exercice 1. Arbres binaires [2 pt + 1 pt]

Question 1.A. Écrivez la définition d'une fonction C **réursive** `nbre_etiquette_AB` qui reçoit en entrées un lien h vers un arbre binaire et un entier v et renvoie le nombre de nœuds de l'arbre pointé par h qui portent une étiquette égale à v .

Corrigé

```
1 int nbre_etiquette_AB(link h, int v) {
2     int res = 0;
3     if (h) {
4         if (h->label == v) ++res;
5         res += nbre_etiquette_AB(h->left, v) + nbre_etiquette_AB(h->right, v);
6     }
7     return res;
8 }
```

Question 1.B. Exprimez le nombre de comparaisons d'entiers effectuées lors de l'exécution de l'appel `nbre_etiquette_AB(h, v)` en fonction de la taille n de l'arbre pointé par h .

Corrigé

Soit n la taille de l'arbre binaire pointé par h et $C(n)$ le nombre de comparaisons d'entiers effectuées lors de l'exécution de l'appel `nbre_etiquettes(h, v)`. On a

$$\begin{cases} C(0) &= 0 \\ C(n+1) &= 1 + C(m) + C(n-m) \quad \text{pour un certain } m, \quad 0 \leq m \leq n \end{cases}$$

On montre alors facilement par récurrence sur n que $C(n) = n$.

Exercice 2. Tas binaire [1 pt + 1 pt + 2 pt]

Avertissement. Dans cet exercice, on suppose que **la priorité est donnée au plus GRAND entier**.

Question 2.A. Soit T un arbre binaire à étiquettes entières. Rappelez les propriétés qui font de T un tas binaire.

Corrigé

T est un tas binaire à valeurs entières pour la priorité donnée au plus grand entier si :

- T est un arbre binaire complet « tassé à gauche », c'est-à-dire que tous les niveaux, sauf éventuellement le dernier, sont complets et que tout nœud de l'avant-dernier niveau, si T a au moins deux niveaux, ne peut avoir de fils gauche que si tous ses frères aînés (les nœuds du même niveau situés à sa gauche) ont chacun un fils droit et un fils gauche d'une part, et ne peut avoir de fils droit que s'il a un fils gauche d'autre part ;
- l'étiquette de chaque nœud de T est supérieure ou égale à celles de ses fils, s'il en a.

Question 2.B On implémente un tas binaire d'au plus vingt entiers à l'aide d'un tableau t dont $t[0]$ contient la taille du tas. En supposant que le tas est initialement vide, représentez le tableau t après qu'on y a inséré successivement les valeurs 33, 77, 43, 71, 36, 88, 12, 11, 6, 9, 58, 0, 67, 0, 15, 50.

t																				
-----	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

Corrigé

Après insertion des 16 entiers, $t[1]$ contient l'étiquette de la racine du tas binaire, et si i est l'indice (dans le tableau t) d'un nœud du tas, les indices de ses fils gauche et droit, s'ils existent, sont $2i$ et $2i + 1$ respectivement.

t	16	88	71	77	50	58	67	15	33	6	9	36	0	43	0	12	11				
-----	----	----	----	----	----	----	----	----	----	---	---	----	---	----	---	----	----	--	--	--	--

Question 2.C Écrivez la définition d'une fonction C `test_tas` qui reçoit en entrées un tableau t d'entiers et un entier n . Elle renvoie 1 si le segment $t[1, \dots, n]$ implémente un tas binaire de n entiers et renvoie 0 sinon.

Corrigé

On vérifie que la propriété fondamentale du tas binaire, à savoir l'étiquette de tout nœud est supérieure ou égales à celles de ses fils, s'ils existent, est respecté pour tous les indices i tels que $1 \leq i \leq \lfloor n/2 \rfloor$.

```
1 int test_tas(int *t, int n) {
2     for(int i = 1; 2*i <= n; ++i) {
3         if (t[i] < t[2*i]) return 0;
4         if (2*i + 1 <= n && t[i] < t[2*i + 1]) return 0;
5     }
6     return 1;
7 }
```

Exercice 3. Arbres binaires de recherche (ABR) [1 pt + 3 pt + 3 pt + 3 pt + 1 pt]

Avertissement. Dans cet exercice, on suppose que **la priorité est donnée au plus GRAND entier**.

Question 3.A. Soit T un arbre binaire à étiquettes entières. Rappelez les propriétés qui font de T un arbre binaire de recherche.

Corrigé

Un arbre binaire T à étiquettes entières est un arbre binaire de recherche (ABR) pour la priorité donnée au plus grand entier si, **quel que soit le nœud de T ,**

- son étiquette est inférieure ou égale à celles de **tous les nœuds** de son sous-arbre gauche,
- son étiquette est supérieure ou égale à celles de **tous les nœuds** de son sous-arbre droit.

Autre définition

Un arbre binaire T à étiquettes entières est un arbre binaire de recherche (ABR) pour la priorité donnée au plus grand entier si,

- T est l'arbre vide

OU

- le sous-arbre gauche de la racine de T est un ABR (pour la même priorité) dont **toutes les étiquettes** sont supérieures ou égales à celles de la racine **ET** le sous-arbre droit de la racine de T est un ABR (pour la même priorité) dont **toutes les étiquettes** sont inférieures ou égales à celles de la racine.

Question 3.B. Écrivez la définition d’une fonction C `fusion_ABR` qui reçoit en entrées deux liens h et k vers des ABR. On suppose que la plus petite étiquette de l’ABR pointé par h est supérieure ou égale à la plus grande étiquette de l’ABR pointé par k . La fonction renvoie un lien vers un ABR stockant l’union des étiquettes stockées dans les ABR pointées par h et k .

N.B. Le nombre d’instructions (tests de nullité et affectations de liens) exécutées lors de l’appel `fusion_ABR(h, k)` doit être $O(m)$ où m est la **hauteur** de l’ABR pointé par k .

Corrigé

Comme on suppose que chaque étiquette de l’ABR pointé par h est supérieure ou égale à toutes les étiquettes de l’ABR pointé par k , il suffit d’accrocher h comme sous-arbre gauche du nœud portant la plus grande étiquette dans l’ABR pointé par k , qui est le nœud « le plus à gauche ». On pourrait aussi accrocher k comme sous-arbre droit du nœud portant la plus petite étiquette dans l’ABR pointé par h , qui est le nœud « le plus à droite » mais cela ne respecterait pas la contrainte de complexité de l’énoncé, qui exige que le nombre d’instructions soit dominé par la hauteur de l’arbre pointé par k .

```

1 link fusion_ABR(link h, link k) {
2     link courant = k;
3     /* N.B. L'instruction suivante n'est pas indispensable mais elle evite */
4     /* de parcourir toute une branche de k si h est un lien vers un arbre vide */
5     if (!h) return k;
6     if (!k) return h;
7     /* On recherche le nœud le plus à gauche dans l'ABR pointé par k */
8     /* pour y accrocher h */
9     while (courant->left) { courant = courant->left; }
10    courant->left = h;
11    return k;
12 }
```

On peut préférer une version récursive de cette fonction de fusion.

```

1 /** Version récursive de fusion_ABR */
2 link fusion_ABR(link h, link k) {
3     if (!h) return k;
4     if (!k) return h; /* cas de base (k arbre vide) */
5     k->left = fusion_ABR(h, k->left);
6     return k;
7 }
```

Pour compenser la déséquilibre introduit en accrochant comme ci-dessus l’ABR pointé par h comme sous-arbre gauche du nœud « maximal » de l’ABR pointé par k , on peut promouvoir ce nœud comme nouvelle racine.

```

1 link fusion_ABR(link h, link k) {
2     link courant = k, pere = NULL;
3     if (!h) return k;
4     if (!k) return h;
5     while (courant->left) {
6         pere = courant;
7         courant = courant->left;
8     }
9     if (pere) {
10        pere->left = courant->right;
11        courant->right = k;
12    }
13    courant->left = h;
14    return courant;
15 }
```


Justification de la borne de complexité

On ne visite que des nœuds de la branche « la plus à gauche » de l'ABR pointé par k avec à chaque visite au plus 1 test de nullité et 2 affectations. En dehors de ces visites, le nombre de tests de nullité et d'affectations de liens est au plus 9. Au total le nombre de tests de nullité et d'affectations est $O(m)$

Question 3.C. Écrivez la définition d'une fonction C `suppression_noeud_ABR` qui reçoit en entrées un lien h vers un ABR et un entier v . Elle renvoie un lien vers l'ABR résultant de la suppression (dans l'ABR pointé par h) d'un nœud portant l'étiquette v , si un tel nœud existe.

N.B. Le nombre d'instructions (comparaisons et affectations de liens) exécutées lors de l'appel `suppression_noeud_ABR(h, v)` doit être $O(m)$ où m est la **hauteur** de l'ABR pointé par h .

Corrigé

On recherche un nœud portant l'étiquette v en descendant le long d'une branche de l'ABR. Si on en trouve un, on le remplace par l'ABR résultant de la fusion de son sous-arbre gauche et de son sous-arbre droit, sans oublier de libérer l'espace mémoire occupé sur le tas par le nœud supprimé..

```
1 link suppression_noeud_ABR(link h, int v) {
2     link nouveau;
3     if (!h) return h;
4     if (v > h->label) {
5         h->left = suppression_noeud_ABR(h->left, v);
6         return h;
7     }
8     if (v < h->label) {
9         h->right = suppression_noeud_ABR(h->right, v);
10        return h;
11    }
12    /* Si h est un lien vers un arbre dont la racine porte l'etiquette v */
13    /* on peut appeler fusion_ABR sur h->left et h->right */
14    /* car chaque étiquette de h->left a une priorite superieure ou egale */
15    /* a chaque etiquette de h->right */
16    nouveau = fusion_ABR(h->left, h->right);
17    free(h);
18    h = nouveau;
19    return h;
20 }
```

Justification de la borne de complexité

Le raisonnement suit les mêmes lignes que pour 3.B :

On descend dans une branche de l'ABR pointé par h à la recherche d'une étiquette égale à v et éventuellement, on poursuit dans la même branche en faisant appel à `fusion_ABR`. On ne visite donc que des noeuds d'une même branche de l'ABR pointé par h avec à chaque visite un nombre uniformément borné de comparaisons et d'affectations. Au total le nombre de comparaisons d'entiers et de tests de nullité et d'affectations de liens est $O(m)$.

Question 3.D. Écrivez la définition d'une fonction **réursive** C `test_bornes_ABR` qui reçoit en entrées un lien h vers un arbre binaire et deux pointeurs a et b vers des entiers. Par convention, si a est le pointeur `NULL`, alors la « valeur » pointée par a est $+\infty$, et si b est le pointeur `NULL`, alors la « valeur » pointée par b est $-\infty$. Dans tous les cas, vous supposerez que la valeur pointée par a est supérieure ou égale à celle pointée par b . La fonction renvoie 1 si l'arbre binaire pointé par h est un ABR dont toutes les étiquettes sont comprises entre les valeurs pointées par b et a , et renvoie 0 sinon.

Corrigé

Si h est un lien vers un arbre vide, alors on renvoie 1 (l'arbre vide est un ABR).

Si h est un lien vers un arbre dont la racine porte une étiquette qui n'est pas comprise entre les « valeurs » de a et b , alors on renvoie 0, sinon on teste récursivement que le sous-arbre gauche de l'arbre pointé par h est un ABR dont toutes les étiquettes sont comprises entre la « valeur » de a et la valeur de l'étiquette de la racine pointée par h d'une part, et que le sous-arbre droit est un ABR dont toutes les étiquettes sont comprises entre l'étiquette de la racine pointée par h et la « valeur » de b d'autre part.

N.B. a (respectivement, b) étant un pointeur vers un entier, quand on compare l'étiquette de la racine de l'arbre pointé par h avec la valeur de a (resp., b), il faut s'assurer que le pointeur n'est pas `NULL`.

```

1 int test_bornes_ABR(link h, int *a, int *b) {
2     if (!h) return 1;
3     if ((a && h->label > *a) || (b && h->label < *b)) return 0;
4     return test_bornes_AB(h->left, a, &(h->label)) && test_bornes_AB(h->right, &(h->label), b);
5 }

```

Question 3.E. Écrivez la définition d'une fonction C `test_ABR` qui reçoit en entrées un lien h vers un arbre binaire. Elle renvoie 1 si l'arbre binaire pointé par h est un ABR et renvoie 0 sinon. Le corps de la fonction ne doit contenir qu'une instruction.

Corrigé

Pour tester si h est un lien vers un ABR, on teste si c'est un lien vers un ABR dont toutes les étiquettes sont comprises entre $+\infty$ et $-\infty$.

```

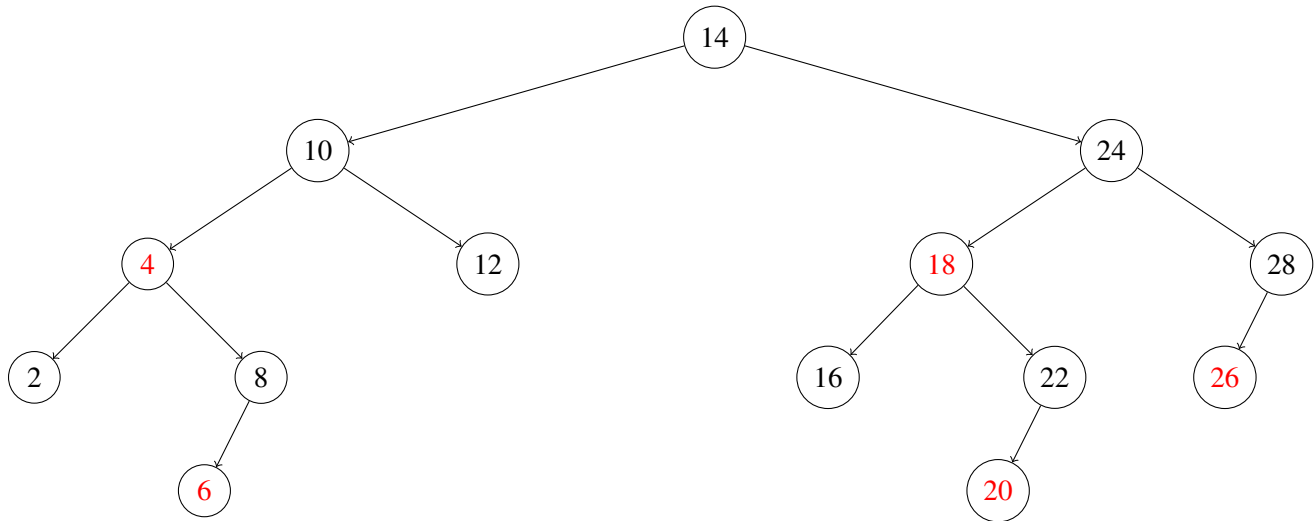
1 int test_ABR(link h) {
2     return test_bornes_ABR(h, NULL, NULL);
3 }

```

Exercice 4. Insertion dans un arbre rouge-noir [1 pt + 2 pt + 1 pt]

Avertissement. Dans cet exercice, on suppose que **la priorité est donnée au plus PETIT entier**.

Soit l'arbre rouge-noir représenté ci-dessous ; ses seuls liens rouges sont les liens $10 \rightarrow 4$, $8 \rightarrow 6$, $24 \rightarrow 18$, $22 \rightarrow 20$ et $28 \rightarrow 26$.



Question 4.A. Rappelez la définition d'un LLRBT (*left-leaning balanced red-black tree*) et montrez que l'arbre binaire représenté ci-dessus en est un.

Corrigé

Un arbre rouge-noir est un LLRBT si

- c'est un arbre binaire de recherche ;
- toutes ses branches ont le même nombre de liens noirs ;
- le long d'une branche, il y a toujours au moins un lien noir entre deux liens rouges ;
- tous les liens rouges lient un nœud à son fils gauche.

On vérifie aisément que l'arbre rouge-noir ci-dessus vérifie toutes ces propriétés.

Question 4.B. Écrivez la suite ordonnée des six opérations élémentaires (« accrochage » comme nouvelle feuille, rotation gauche, rotation droite ou échange de couleurs) entraînées par l'insertion d'un sommet d'étiquette 23 dans le LLRBT représenté ci-dessus, en précisant pour chaque opération le sommet concerné.

Corrigé

La suite d'opérations élémentaires est la suivante.

1. Insertion de 23 comme fils droit rouge de 22.
2. *Colour-flip* des liens autour de 22.
3. Rotation gauche de 18.
4. Rotation droite de 24.
5. *Colour-flip* des liens autour de 22.
6. Rotation gauche de 14.

Question 4.C. Dessinez dans l'espace blanc ci-dessous l'arbre résultant de l'insertion d'un sommet d'étiquette 23 dans le LLRBT de la page 14, en indiquant les liens rouges par un trait de couleur ou un trait nettement plus épais.

Corrigé

Voici l'arbre après l'insertion d'un nœud d'étiquette 23 ; les liens rouges sont les liens $10 \rightarrow 4$, $8 \rightarrow 6$, $22 \rightarrow 14$ et $28 \rightarrow 26$.

